

Lab 3 - Мікросервиси з використанням Hazelcast Distributed Map

Лінк на гітхаб: https://github.com/Sirska-Mariia/homework1/tree/micro_hazelcast

Запустити три екземпляри **logging-service** (кожен з екземплярів запускається у власному Docker-контейнері),

Відповідно має запуститись також кластер Hazelcast з трьох серверів

На скріншоті видно, що запущено три окремі екземпляри logging-service, кожен у власному контейнері: logging-node-1 (порт 8081), logging-node-2 (порт 8082), logging-node-3 (порт 8083)

Також видно, що кластер Hazelcast успішно запустився з трьох серверів:

banking_system_-hazelcast-1, banking_system_-hazelcast-2,

banking_system_-hazelcast-3

```
PS C:\апз\homework1\banking_system> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
PORTS         NAMES
31e9367cee51   my-facade-service                  "uvicorn main:app --..." 6 minutes ago  Up 6 minutes
0.0.0.0:8080->8000/tcp   facade-app
026804c0dc39   my-logging-service                 "uvicorn main:app --..." 6 minutes ago  Up 6 minutes
0.0.0.0:8082->8000/tcp   logging-node-2
02254d3a69a1   my-logging-service                 "uvicorn main:app --..." 6 minutes ago  Up 6 minutes
0.0.0.0:8083->8000/tcp   logging-node-3
27ea02b2a1e2   my-counter-service                 "uvicorn main:app --..." 6 minutes ago  Up 6 minutes
0.0.0.0:8084->8000/tcp   counter-app
2392aeb343e4   my-logging-service                 "uvicorn main:app --..." 6 minutes ago  Up 6 minutes
0.0.0.0:8081->8000/tcp   logging-node-1
5c91fe90c66b   hazelcast/hazelcast:5.3.0         "hz start"              6 minutes ago  Up 6 minutes
5701/tcp             banking_system_-hazelcast-2
8ddad12b64b2   hazelcast/hazelcast:5.3.0         "hz start"              6 minutes ago  Up 6 minutes
5701/tcp             banking_system_-hazelcast-3
981e12ce18a8   hazelcast/hazelcast:5.3.0         "hz start"              6 minutes ago  Up 6 minutes
5701/tcp             banking_system_-hazelcast-1
0f6f9aa72730   postgres:15                        "docker-entrypoint.s..." 11 minutes ago  Up 11 minutes
0.0.0.0:5432->5432/tcp   bank-db
PS C:\апз\homework1\banking_system>
```

Через HTTP POST записати 10 транзакцій msg1-msg10 через **facade-service**

```
[+] Restarting 1/1
✓ Container counter-app Started 0.3s
PS C:\анз\homework1\banking_system_> 1..10 | ForEach-Object {
>>
>>     $msg = "msg" + $_
>>
>>     $body = @{
>>
>>         user_id = "Mariia"
>>
>>         amount = [int]$_
>>
>>         message = $msg
>>
>>     } | ConvertTo-Json
>>
>>
>>     Invoke-RestMethod -Uri "http://localhost:8080/transaction" -Method Post -Body $body -ContentTy
pe "application/json"
>>
>>
>>     Write-Host "Надіслано: $msg"
>>
>> }
```

На скріншоті видно процес надсилання 10 послідовних транзакцій за допомогою PowerShell-скрипта. Кожен запит є стандартним HTTP POST запитом на кінцеву точку /transaction сервісу facade-service. У відповіді від сервера видно унікальний transaction_id для кожного запису та актуальний баланс користувача, який зростає від 1.0 до 55.0.

```
Надіслано: msg1
Надіслано: msg2
Надіслано: msg3
Надіслано: msg4
Надіслано: msg5
transaction_id          balance
-----
aba8cddc-f58e-4095-945d-e5a63575fb5b    1.0
eee5370d-2828-4a67-9ba4-3f9bc212445a    3.0
e0d32024-84f0-4a65-890d-e5c70de5bbce    6.0
77720441-6bf5-46f2-ae34-f6113c6a6b14   10.0
b1f66cd6-3b16-496c-b9c1-ca0e5849a190   15.0
ef193965-e9eb-4724-b0d1-5db952dd94ed   21.0
Надіслано: msg6
7437b6ac-b547-41f9-adec-4cd8e12f3bec   28.0
Надіслано: msg7
e4f703bf-9b0a-4a4f-bcff-76fa6957b9eb   36.0
Надіслано: msg8
1a5026cd-bd43-4559-b071-94c0a836f656   45.0
Надіслано: msg9
150550be-5b9d-4f0a-8303-ceeacb9b4c92   55.0
Надіслано: msg10
```

Показати які повідомлення отримав кожен з екземплярів **logging-service** (це має бути видно у логах сервісу)

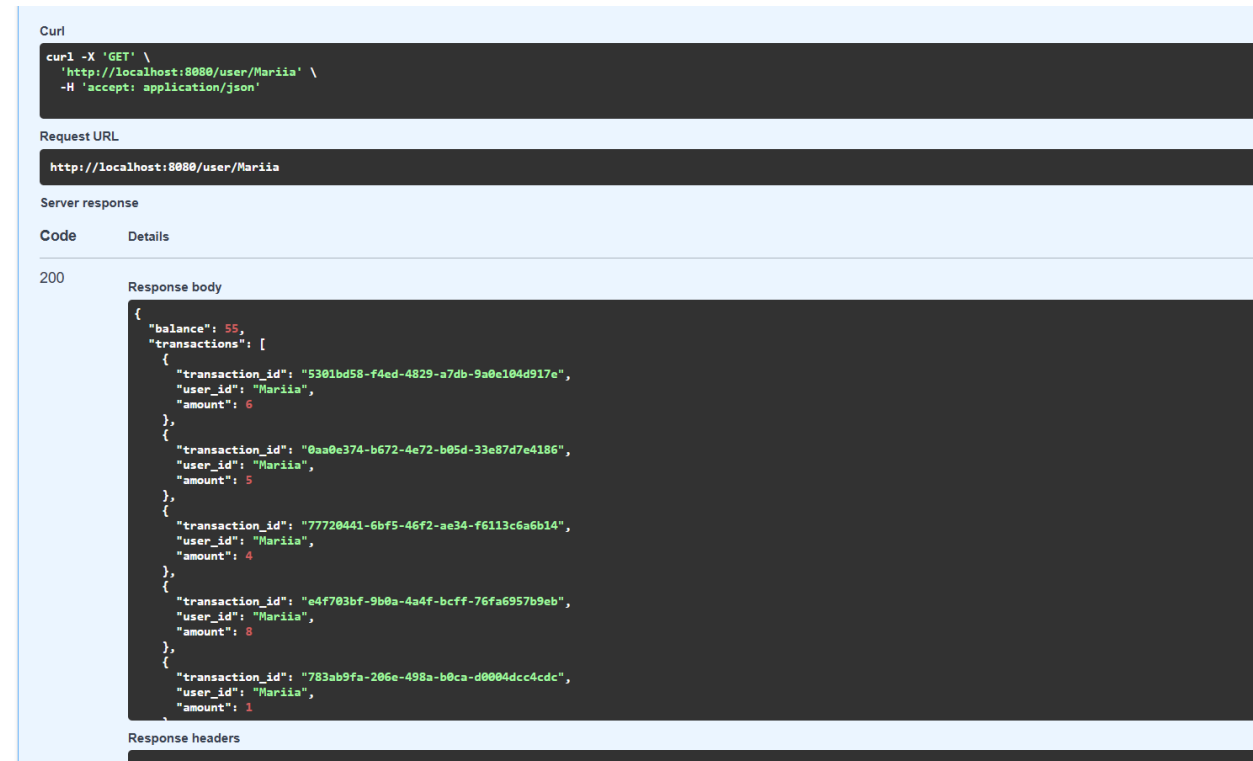
```

PS C:\апз\homework1\banking_system> # Тільки для лог-сервісів (PowerShell)
PS C:\апз\homework1\banking_system> docker-compose logs logging-1 logging-2 logging-3
logging-node-2 | INFO:      Started server process [1]
logging-node-1 | INFO:      Started server process [1]
logging-node-1 | INFO:      Waiting for application startup.
logging-node-1 | INFO:      Application startup complete.
logging-node-1 | INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
logging-node-1 | [logging-1] Logged: 51c62444-cf0e-44d3-955b-323aaf57385f for user Mariia
logging-node-1 | INFO:      172.19.0.10:51148 - "POST /log HTTP/1.1" 200 OK
logging-node-1 | [logging-1] Logged: 5fa35922-e473-4256-86a8-f94509393898 for user Mariia
logging-node-1 | INFO:      172.19.0.10:51148 - "POST /log HTTP/1.1" 200 OK
logging-node-1 | [logging-1] Logged: 4780047d-1973-43f8-9ea2-9061d2b15c43 for user Mariia
logging-node-1 | INFO:      172.19.0.10:51148 - "POST /log HTTP/1.1" 200 OK
logging-node-1 | [logging-1] Logged: a4b2f92c-9f45-42ad-96e7-9eb5f8a006ab for user Mariia
logging-node-1 | INFO:      172.19.0.10:51148 - "POST /log HTTP/1.1" 200 OK
logging-node-1 | [logging-1] Logged: c3abd015-425e-4c28-9a4f-6c758b7bbf8d for user Mariia
logging-node-1 | INFO:      172.19.0.10:51148 - "POST /log HTTP/1.1" 200 OK
logging-node-2 | INFO:      Waiting for application startup.
logging-node-2 | INFO:      Application startup complete.
logging-node-2 | INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
logging-node-2 | [logging-2] Logged: dfabfd02-1821-41f3-964a-0f8f3865a090 for user Mariia
logging-node-2 | INFO:      172.19.0.10:49424 - "POST /log HTTP/1.1" 200 OK
logging-node-3 | INFO:      Started server process [1]
logging-node-3 | INFO:      Waiting for application startup.
logging-node-3 | INFO:      Application startup complete.
logging-node-3 | INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
logging-node-3 | [logging-3] Logged: 0e75636a-9569-43e8-872d-38c5368ef273 for user Mariia
logging-node-3 | INFO:      172.19.0.10:57026 - "POST /log HTTP/1.1" 200 OK
logging-node-3 | [logging-3] Logged: 46f56b9b-e758-4de9-a8db-09e9a3654ddf for user Mariia
logging-node-3 | INFO:      172.19.0.10:57026 - "POST /log HTTP/1.1" 200 OK
logging-node-3 | INFO:      172.19.0.10:33836 - "GET /logs/Mariia HTTP/1.1" 200 OK
logging-node-2 | [logging-2] Logged: 5dbe9715-cb65-4083-9f60-b2eb7edb977f for user Mariia
logging-node-2 | INFO:      172.19.0.10:49424 - "POST /log HTTP/1.1" 200 OK
logging-node-2 | [logging-2] Logged: 112b2c38-5ea4-42e0-9145-9407d588095e for user Mariia
logging-node-2 | INFO:      172.19.0.10:49424 - "POST /log HTTP/1.1" 200 OK
PS C:\апз\homework1\banking_system>

```

На скріншоті видно, що запити від facade-app розподіляються між трьома вузлами логування. Наприклад, транзакцію 51c6... обробив вузол 1, а 0e75... обробив вузол 3.

Через HTTP GET з facade-service прочитати транзакції



Виконано перевірку цілісності даних через HTTP GET запит до facade-service за адресою /user/Mariia. Отримана відповідь у форматі JSON містить повний список транзакцій (msg1-msg10).

Перевірка відмовостійкості:

*Вимкнути один/два екземпляри **logging-service** та перевірити чи зможемо записати та прочитати транзакції*

Вимкнення одного екземпляру:

```
PS C:\апз\homework1\banking_system_> docker stop logging-node-1
logging-node-1
PS C:\апз\homework1\banking_system_> $body = @{user_id="Mariia";
e"} | ConvertTo-Json
>> Invoke-RestMethod -Uri "http://localhost:8080/transaction" -Me
```

Додавання кількох нових транзакцій:

```

PS C:\an3\homework1\banking_system> $body = @{user_id="Mariia"; amount=10; message="msg_after_failur
e"} | ConvertTo-Json
>> Invoke-RestMethod -Uri "http://localhost:8080/transaction" -Method Post -Body $body -ContentType "
application/json"

transaction_id          balance
-----
d9180575-44db-4773-bcc3-1a42ac2a87b5  175.0

PS C:\an3\homework1\banking_system> $body = @{user_id="Mariia"; amount=10; message="msg_after_failur
e"} | ConvertTo-Json
>> Invoke-RestMethod -Uri "http://localhost:8080/transaction" -Method Post -Body $body -ContentType "
application/json"

transaction_id          balance
-----
67af47fe-205b-415c-8f57-7c87e5e18a85  185.0

PS C:\an3\homework1\banking_system> $body = @{user_id="Mariia"; amount=10; message="msg_after_failur
e"} | ConvertTo-Json
>> Invoke-RestMethod -Uri "http://localhost:8080/transaction" -Method Post -Body $body -ContentType "
application/json"

```

```

logging-node-1 exited with code 0
logging-node-2 | [logging-2] Logged: d9180575-44db-4773-bcc3-1a42ac2a87b5 for user Mariia
logging-node-2 | INFO: 172.19.0.10:38642 - "POST /log HTTP/1.1" 200 OK
counter-app | INFO: 172.19.0.10:34064 - "POST /update HTTP/1.1" 200 OK
facade-app | INFO: 172.19.0.1:54144 - "POST /transaction HTTP/1.1" 200 OK
logging-node-2 | [logging-2] Logged: 67af47fe-205b-415c-8f57-7c87e5e18a85 for user Mariia
logging-node-2 | INFO: 172.19.0.10:37894 - "POST /log HTTP/1.1" 200 OK
counter-app | INFO: 172.19.0.10:54208 - "POST /update HTTP/1.1" 200 OK
facade-app | INFO: 172.19.0.1:49742 - "POST /transaction HTTP/1.1" 200 OK
counter-app | INFO: 172.19.0.10:54208 - "POST /update HTTP/1.1" 200 OK
logging-node-3 | [logging-3] Logged: 3c3edc9f-64f6-454e-94e7-009db639f03c for user Mariia
logging-node-3 | INFO: 172.19.0.10:59406 - "POST /log HTTP/1.1" 200 OK
facade-app | Node http://logging-1:8000 is down, trying next...
facade-app | INFO: 172.19.0.1:49742 - "POST /transaction HTTP/1.1" 200 OK

```

На скріншоті зафіксовано тест системи на відмовостійкість. Вгорі видно статус logging-node-1 exited with code 0, цей екземпляр було зупинено вручну. При надсиланні наступного запиту facade-app виявив недоступність першої ноди (рядок Node http://logging-1:8000 is down, trying next...). Система автоматично перенаправила запит на logging-node-3, який успішно обробив транзакцію (200 OK). Отже, вихід з ладу окремих компонентів не впливає на загальну працездатність системи.

Вимкнення ще одного екземпляру:

```
PS C:\анз\homework1\banking_system> docker stop logging-node-3
logging-node-3
PS C:\анз\homework1\banking_system> $body = @{user_id="Mariia"; amount=10; message="msg_after_failur
e"} | ConvertTo-Json
>> Invoke-RestMethod -Uri "http://localhost:8080/transaction" -Method Post -Body $body -ContentType "
application/json"
```

```
logging-node-3 exited with code 0
logging-node-2 | [logging-2] Logged: f57221bc-c143-4fa5-a97b-bdd26228739d for user Mariia
logging-node-2 | INFO: 172.19.0.10:50964 - "POST /log HTTP/1.1" 200 OK
counter-app | INFO: 172.19.0.10:38134 - "POST /update HTTP/1.1" 200 OK
facade-app | INFO: 172.19.0.1:54972 - "POST /transaction HTTP/1.1" 200 OK
logging-node-2 | [logging-2] Logged: f0b32486-a544-40f6-930e-7b3804b6d92a for user Mariia
logging-node-2 | INFO: 172.19.0.10:50964 - "POST /log HTTP/1.1" 200 OK
counter-app | INFO: 172.19.0.10:38134 - "POST /update HTTP/1.1" 200 OK
facade-app | INFO: 172.19.0.1:54972 - "POST /transaction HTTP/1.1" 200 OK
logging-node-2 | [logging-2] Logged: cf26deb7-6af9-4496-af0f-dbe303583dca for user Mariia
logging-node-2 | INFO: 172.19.0.10:50964 - "POST /log HTTP/1.1" 200 OK
counter-app | INFO: 172.19.0.10:38134 - "POST /update HTTP/1.1" 200 OK
facade-app | INFO: 172.19.0.1:54972 - "POST /transaction HTTP/1.1" 200 OK
counter-app | INFO: 172.19.0.10:57694 - "POST /update HTTP/1.1" 200 OK
logging-node-2 | [logging-2] Logged: 829d12c6-ae21-41ae-a81d-4fd2a58d6cd5 for user Mariia
logging-node-2 | INFO: 172.19.0.10:60558 - "POST /log HTTP/1.1" 200 OK
facade-app | Node http://logging-3:8080 is down, trying next...
facade-app | INFO: 172.19.0.1:55072 - "POST /transaction HTTP/1.1" 200 OK
logging-node-2 | [logging-2] Logged: 0bf125b5-d379-4adc-888c-8e172dcf525d for user Mariia
```

Вузол logging-node-3 також було зупинено. Одразу після цього на шлюз системи було надіслано нові транзакції з повідомленням msg_after_failure. Так само, як і з одним вимкненим екземпляром logging-service, сервіс facade-app при спробі логування виявив недоступність третьої ноди. Запит було перенаправлено на робочу ноду logging-node-2.

Також, перевірка читання транзакцій:

```
PS C:\анз\homework1\banking_system> Invoke-RestMethod -Uri "http://localhost:8080/user/Mariia" -Meth
od Get

balance transactions
-----
280.0 {@{transaction_id=bfac644d-8e95-45a2-8e95-15051833d47b; user_id=Mariia; amount=3}, @{{tran...
```

```

counter-app | INFO: 172.19.0.10:55076 - "GET /balance/Mariia HTTP/1.1" 200 OK
logging-node-2 | INFO: 172.19.0.10:58354 - "GET /logs/Mariia HTTP/1.1" 200 OK
facade-app | Node http://logging-3:8000 is down, trying next...
facade-app | INFO: 172.19.0.1:42884 - "GET /user/Mariia HTTP/1.1" 200 OK
counter-app | INFO: 172.19.0.10:53706 - "GET /balance/Mariia HTTP/1.1" 200 OK
logging-node-2 | INFO: 172.19.0.10:36850 - "GET /logs/Mariia HTTP/1.1" 200 OK
facade-app | Node http://logging-1:8000 is down, trying next...
facade-app | INFO: 172.19.0.1:58064 - "GET /user/Mariia HTTP/1.1" 200 OK

```

facade-app продовжує успішно обробляти GET-запити (200 OK). Оскільки частина сервісів логування також була вимкнена, фасад автоматично перемикається між доступними адресами (Node is down, trying next...), поки не знаходить працюючий logging-node-2.

Вимкнути один/два ноди Hazelcast та перевірити чи зможемо записати та прочитати транзакції

Відключення двох нод:

```

PS C:\анз\homework1\banking_system_> docker stop banking_system_-hazelcast-1 banking_system_-hazelcast-2
banking_system_-hazelcast-1
banking_system_-hazelcast-2
PS C:\анз\homework1\banking_system_> $body = @{user_id="Mariia"; amount=5; message="hazelcast_fault_test"} | ConvertTo-Json
>> Invoke-RestMethod -Uri "http://localhost:8080/transaction" -Method Post -Body $body -ContentType "application/json"

transaction_id          balance
-----
571bac6c-f6c3-467c-850c-fb5e7c2e04dd  285.0

```

```

hazelcast-1 exited with code 143
hazelcast-2 exited with code 143

```



```

hazelcast-3      | 2026-03-22 23:41:33,474 [ WARN] [hz.pensive_goldstine.cached.thread-3] [c.h.i.s.t.T
cpServerConnectionErrorHandler]: [172.19.0.5]:5701 [dev] [5.3.0] Removing connection to endpoint [172
.19.0.9]:5701 Cause => java.io.IOException {Host is unreachable to address /172.19.0.9:5701}, Error-C
ount: 5
hazelcast-3      | 2026-03-22 23:41:33,479 [ WARN] [hz.pensive_goldstine.cached.thread-3] [c.h.i.c.i.M
embershipManager]: [172.19.0.5]:5701 [dev] [5.3.0] Member [172.19.0.9]:5701 - ae005b2f-daa5-4623-8555
-c359c49ebe77 is suspected to be dead for reason: No connection
hazelcast-3      | 2026-03-22 23:41:33,480 [ WARN] [hz.pensive_goldstine.cached.thread-5] [c.h.i.c.i.M
embershipManager]: [172.19.0.5]:5701 [dev] [5.3.0] Member [172.19.0.2]:5701 - be080cfd-fbf2-4607-bed1
-01832ecfea09 is suspected to be dead for reason: No connection
hazelcast-3      | 2026-03-22 23:41:33,480 [ INFO] [hz.pensive_goldstine.cached.thread-5] [c.h.i.c.i.M
embershipManager]: [172.19.0.5]:5701 [dev] [5.3.0] Starting mastership claim process...

```

```

actionManagerService]: [172.19.0.5]:5701 [dev] [5.3.0] Committing/rolling-back live transactions of [
172.19.0.9]:5701, UUID: ae005b2f-daa5-4623-8555-c359c49ebe77
hazelcast-3      | 2026-03-22 23:41:33,511 [ INFO] [hz.pensive_goldstine.cached.thread-3] [c.h.i.c.Clus
terService]: [172.19.0.5]:5701 [dev] [5.3.0]
hazelcast-3      |
hazelcast-3      | Members {size:1, ver:4} [
hazelcast-3      |     Member [172.19.0.5]:5701 - d97a8b78-817e-404f-abcb-49830e999377 this
hazelcast-3      | ]
hazelcast-3      | 2026-03-22 23:41:33,511 [ INFO] [hz.pensive_goldstine.cached.thread-3] [c.h.i.c.i.M
embershipManager]: [172.19.0.5]:5701 [dev] [5.3.0] Mastership is claimed with: MembersView{version=4,
members=[MemberInfo{address=[172.19.0.5]:5701, uuid=d97a8b78-817e-404f-abcb-49830e999377, cpMemberUU
ID=null, liveMembers=false, memberIsInitiatedVersion=211}

```

Нода hazelcast-3 зрозуміла, що вона залишилася одна, перерахувала стан кластера і оголосила себе головною (Mastership is claimed). Вона взяла на себе відповідальність за всі дані, які раніше були розподілені між трьома вузлами.

```

hazelcast-3      | [172.19.0.9]:5701 - ae005b2f-daa5-4623-8555-c359c49ebe77
hazelcast-3      | [172.19.0.2]:5701 - be080cfd-fbf2-4607-bed1-01832ecfea09
hazelcast-3      | }
hazelcast-3      | 2026-03-22 23:41:33,847 [ INFO] [hz.pensive_goldstine.migration] [c.h.i.p.i.Migrati
onManager]: [172.19.0.5]:5701 [dev] [5.3.0] Partition balance is ok, no need to repartition.
logging-node-2   | [logging-2] Logged: 571bac6c-f6c3-467c-850c-fb5e7c2e04dd for user Mariia
logging-node-2   | INFO:      172.19.0.10:40592 - "POST /log HTTP/1.1" 200 OK
facade-app       | Node http://logging-3:8000 is down, trying next...

```

Hazelcast підтвердив, що всі частини даних на місці. Попри те, що 2 ноди вимкнені, остання нода має всі необхідні копії даних.

```

hazelcast-3      | [172.19.0.9]:5701 - ae005b2f-daa5-4623-8555-c359c49ebe77
hazelcast-3      | [172.19.0.2]:5701 - be080cfd-fbf2-4607-bed1-01832ecfea09
hazelcast-3      | }
hazelcast-3      | 2026-03-22 23:41:33,847 [ INFO] [hz.pensive_goldstine.migration] [c.h.i.p.i.Migrati
onManager]: [172.19.0.5]:5701 [dev] [5.3.0] Partition balance is ok, no need to repartition.
logging-node-2   | [logging-2] Logged: 571bac6c-f6c3-467c-850c-fb5e7c2e04dd for user Mariia
logging-node-2   | INFO:      172.19.0.10:40592 - "POST /log HTTP/1.1" 200 OK

```

Після того, як кластер Hazelcast перезібрався, відправлено новий запит. Система успішно його прийняла, записала лог і оновила базу.

Перевірка читання транзакцій:

```
PS C:\анз\homework1\banking_system_> Invoke-RestMethod -Uri "http://localhost:8080/user/Mariia" -Method Get

balance transactions
-----
285.0 @{transaction_id=e99bc7f8-2910-47a6-a23c-116174168173; user_id=Mariia; amount=9}, @{{tran...

PS C:\анз\homework1\banking_system_>

logging-node-2 | INFO:      172.19.0.10:36388 - "GET /logs/Mariia HTTP/1.1" 200 OK
counter-app    | INFO:      172.19.0.10:47978 - "GET /balance/Mariia HTTP/1.1" 200 OK
facade-app     | INFO:      172.19.0.1:44204 - "GET /user/Mariia HTTP/1.1" 200 OK
```

На скріншотах продемонстровано успішне виконання запиту на читання даних у стані, коли в кластері Hazelcast залишився лише один активний вузол.

Тестування продуктивності

```
Результати Сценарій 1 (Різні рахунки):
Загальний час: 2921.88 сек
RPS (запитів/сек): 34.22
Сер. час виклику Logging: 0.2422 сек
Сер. час виклику Counter: 0.2422 сек
```

```
Результати Сценарій 2 (Один рахунок):
Загальний час: 3329.85 сек
RPS (запитів/сек): 30.03
Загальний час: 3329.85 сек
RPS (запитів/сек): 30.03
Сер. час виклику Logging: 0.2211 сек
Сер. час виклику Counter: 0.2211 сек
```

	Сценарій 1 (Різні рахунки)	Сценарій 2 (Один рахунок)
--	----------------------------	---------------------------

RPS (запитів/сек)	34.22	30.03
Загальний час (100к зап.)	2921.88 сек	3329.85 сек
Сер. час Logging	~0.2422 сек	~0.2211 сек
Сер. час Counter	~0.2422 сек	~0.2211 сек

Для перевірки системи було використано асинхронний Python-клієнт, що імітує роботу 10 клієнтів одночасно. Кожен сценарій передбачав сумарно 100,000 транзакцій.

Сценарій 1: Найвищий показник RPS (34.22).

Оскільки кожен клієнт працював зі своїм унікальним `user_id`, база даних PostgreSQL могла оновлювати різні рядки таблиці одночасно без очікування блокувань. Це дозволило системі максимально ефективно використовувати ресурси процесора та мережі.

Сценарій 2: зниження продуктивності на ~14% порівняно з першим сценарієм.

Усі 10 клієнтів одночасно надсилали запити на зміну одного й того самого рядка в базі даних (`target_user`), що спричинило чергу на рівні Row-Level Locking у PostgreSQL. Система змушена була обробляти ці оновлення послідовно, щоб уникнути помилок розрахунку, і це призвело до збільшення загального часу виконання на 408 секунд.

Порівняння отриманих результатів з результатами з першого завдання:

Результати з першого завдання:

```
Запуск: Сценарій 1 (Різні рахунки)
Налаштування: 10 клієнтів по 10000 запитів
Результати Сценарій 1 (Різні рахунки):
  Загальний час: 1797.23 сек
  RPS (запитів/сек): 55.64
  Сер. час виклику Logging: 0.1236 сек
  Сер. час виклику Counter: 0.1236 сек

Запуск: Сценарій 2 (Один рахунок)
Налаштування: 10 клієнтів по 10000 запитів
Результати Сценарій 2 (Один рахунок):
  Загальний час: 2061.00 сек
  RPS (запитів/сек): 48.52
  Сер. час виклику Logging: 0.1148 сек
  Сер. час виклику Counter: 0.1148 сек
```

Метрика	Сценарій 1 (Різні рахунки)	Сценарій 2 (Один рахунок)
Загальний час виконання	1797.23 сек	2061.00 сек
RPS (запитів за секунду)	55.64	48.52
Сер. час виклику Logging	0.1236 сек	0.1148 сек
Сер. час виклику Counter	0.1236 сек	0.1148 сек

Пропускна здатність у третій лабі впала приблизно в 3-4 рази порівняно з першою, оскільки у Лабі 1 операції виконувалися в оперативній пам'яті одного процесу. У Лабі 3 кожен запит проходить через каскад контейнерів Docker.

Додатковий час витрачається на запис транзакцій у PostgreSQL на диск та синхронізацію даних у кластері Hazelcast.

Середній час виклику сервісів зріс до ~0.11-0.12 сек, що підтверджує, що мікросервісна архітектура завжди повільніша за монолітну через витрати на серіалізацію JSON та мережеву взаємодію.

Хоча система стала повільнішою, вона отримала властивості, яких не було в першій лабі: витримує падіння нод дані не зникають після перезапуску.